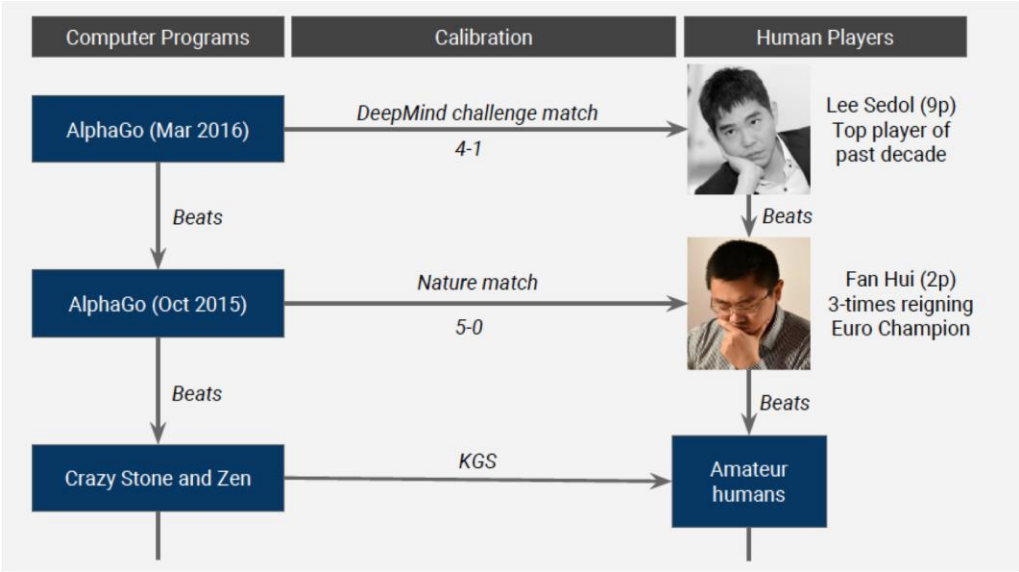
CSCI218: Foundations of Artificial Intelligence

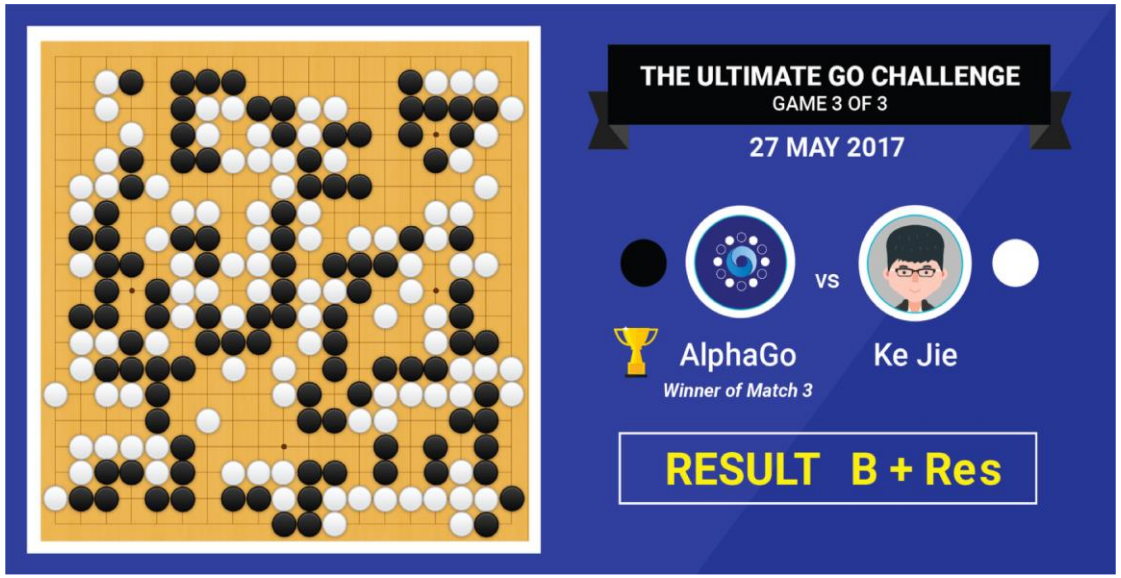
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

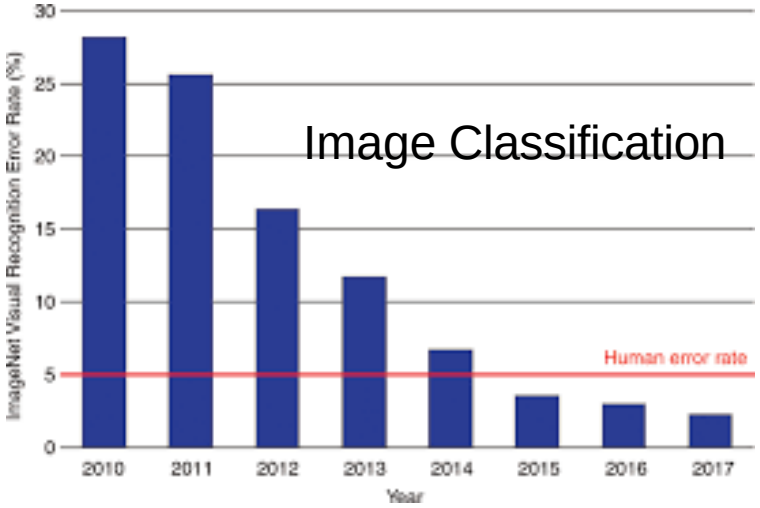
Classical stats/ML: Minimize loss function

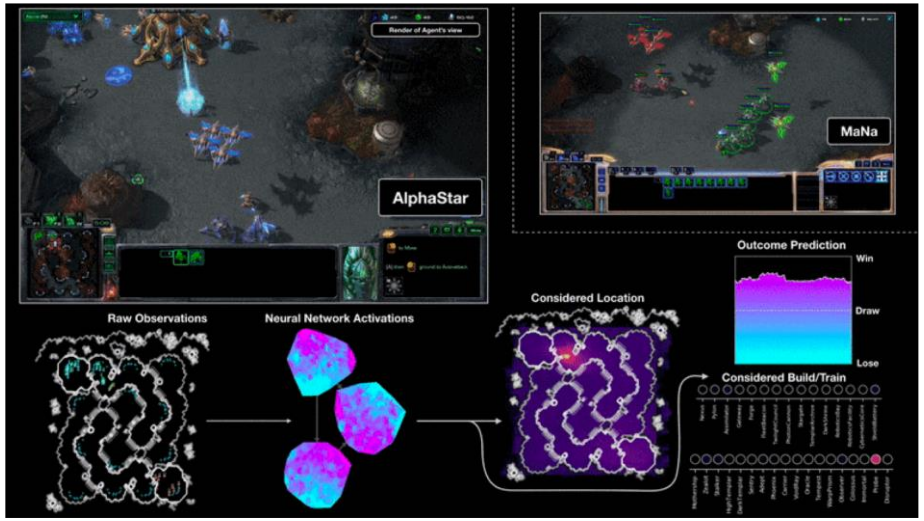
- Which hypothesis space H to choose?
  - *E.g., linear combinations of features*: $h_w(x) = w^T x$
- How to measure degree of fit?
  - *Loss function, e.g., squared error* $\Sigma_j (y_j - w^T x)^2$
- How to trade off degree of fit vs. complexity?
  - *Regularization: complexity penalty, e.g.,* $||w||^2$
- How do we find a good h?
  - *Optimization (closed-form, numerical); discrete search*
- How do we know if a good h will predict well?
  - *Try it and see (cross-validation, bootstrap, etc.)*

Deep Learning/Neural Network

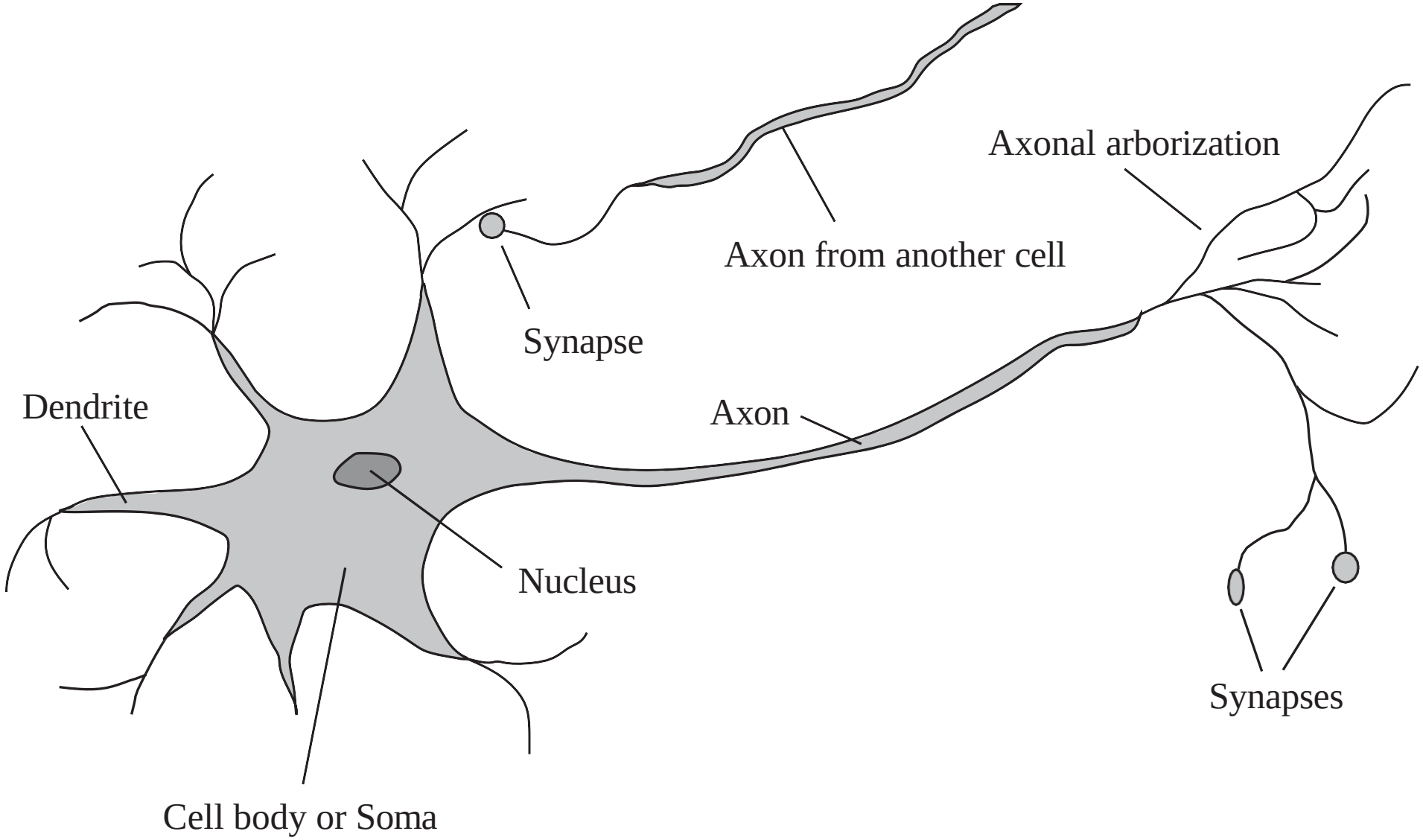




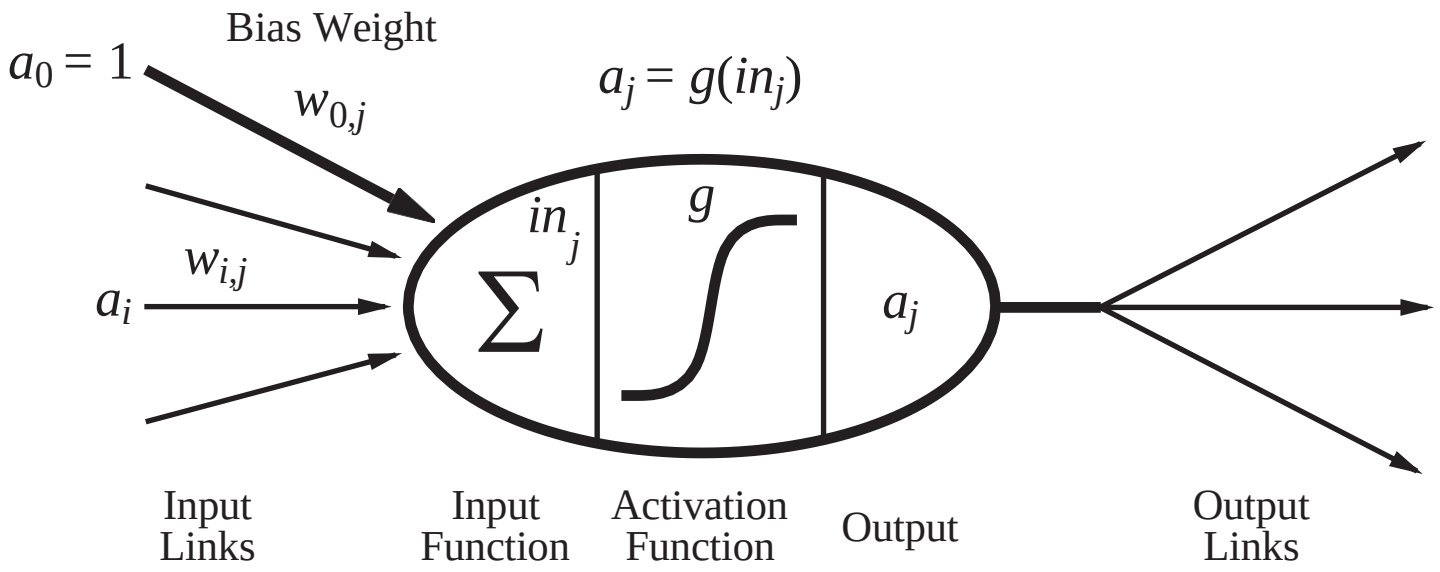




Very loose inspiration: Human neurons

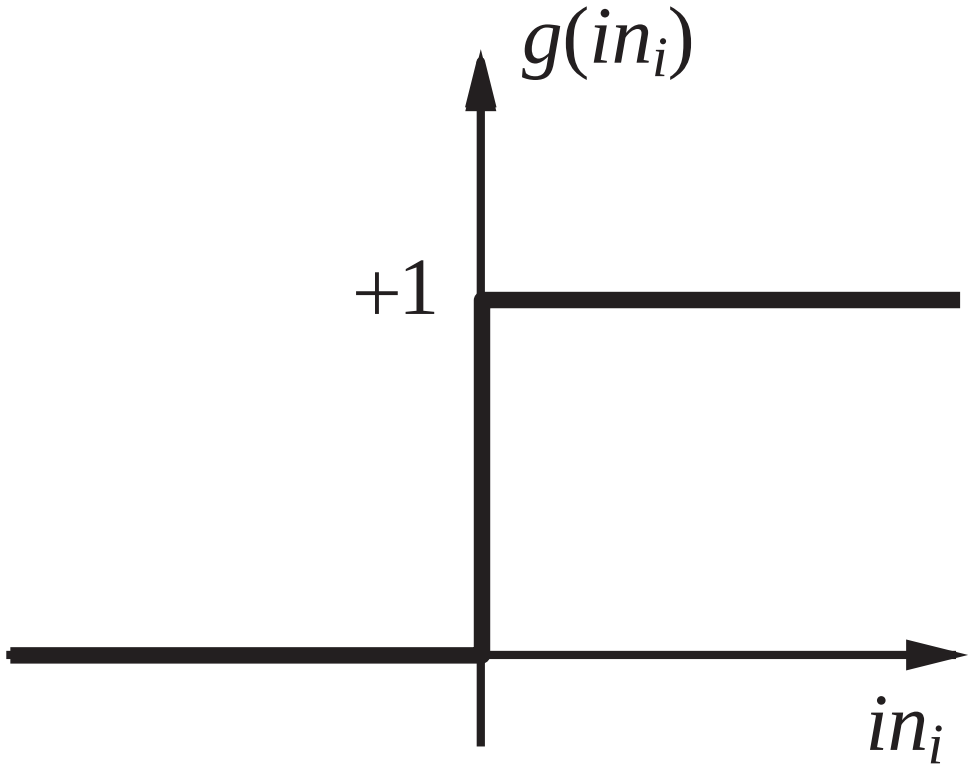


Simple model of a neuron (McCulloch & Pitts, 1943)

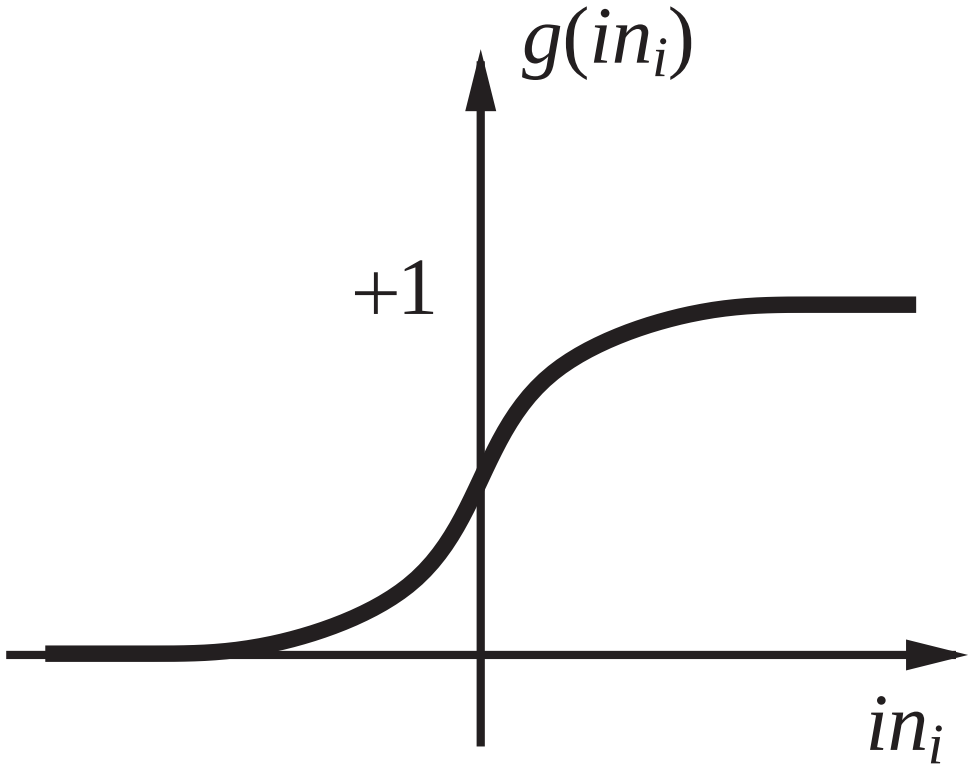


- Inputs a_i come from the output of node i to this node j (or from "outside")
- Each input link has a ***weight*** $w_{i,j}$
- There is an additional fixed input a_0 with ***bias*** weight $w_{0,j}$
- The total input is $in_j = \Sigma_i w_{i,j} a_i$
- The output is $a_j = g(in_j) = g(\Sigma_i w_{i,j} a_i) = g(\boldsymbol{w}.\boldsymbol{a})$

Activation functions g



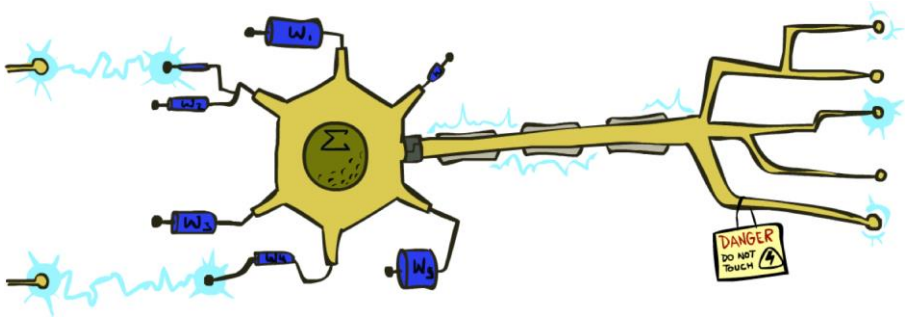
Threshold



Sigmoid $1/(1+e^{-x})$

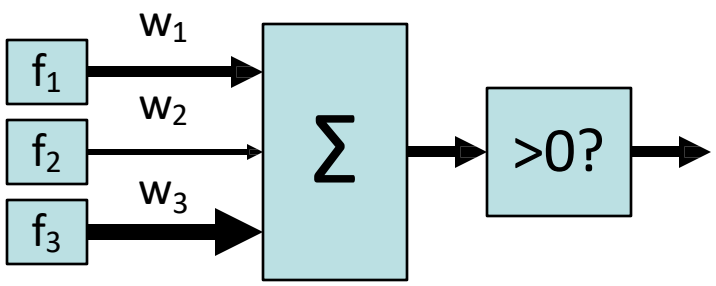
Reminder: Linear Classifiers

- Inputs are feature values
- Each feature has a weight
- Sum is the activation



$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

- If the activation is:
  - Positive, output +1
  - Negative, output -1



How to get probabilistic decisions?

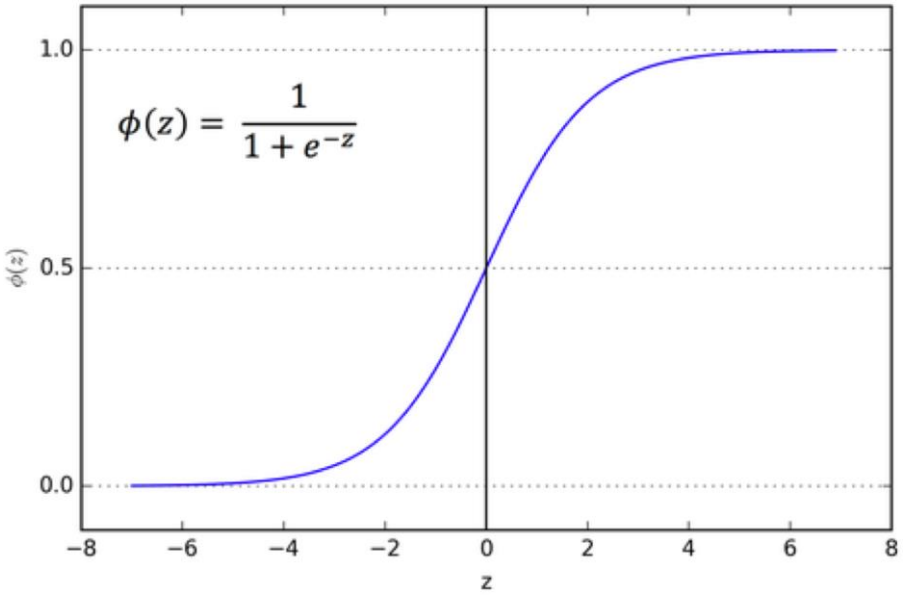
$$z = w \cdot f(x)$$

If $z = w \cdot f(x)$ very positive, want probability going to 1

If $z = w \cdot f(x)$ very negative, want probability going to 0

Sigmoid function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



Best w?

Maximum likelihood estimation:

$$\max_w \quad ll(w) = \max_w \quad \sum_i \log P(y^{(i)}|x^{(i)}; w)$$

with:

$$P(y^{(i)} = +1|x^{(i)}; w) = \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

$$P(y^{(i)} = -1|x^{(i)}; w) = 1 - \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

= Logistic Regression

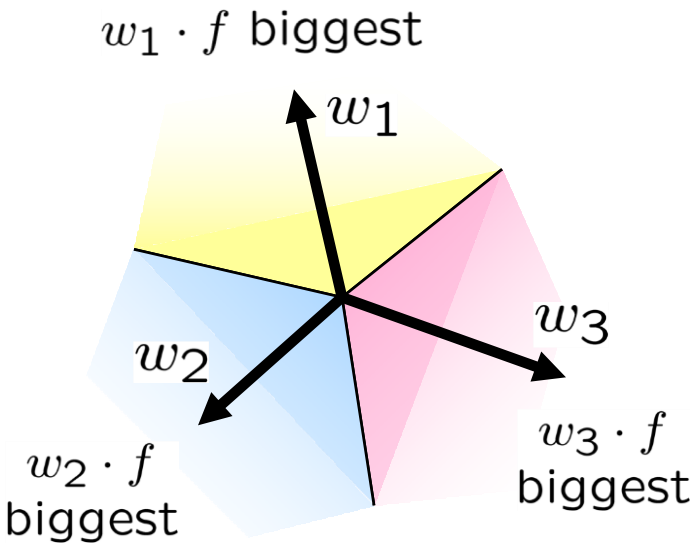
Multiclass Logistic Regression

Multi-class linear classification

A weight vector for each class: w_y

Score (activation) of a class y: $w_y \cdot f(x)$

Prediction w/highest score wins: $y = \arg\max_y \; w_y \cdot f(x)$



How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}, \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}, \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}}_{\text{softmax activations}}$$

Best w?

Maximum likelihood estimation:

$$\max_w \quad ll(w) = \max_w \quad \sum_i \log P(y^{(i)}|x^{(i)}; w)$$

with:

$$P(y^{(i)}|x^{(i)}; w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

= Multi-Class Logistic Regression

Optimization

Optimization

i.e., how do we solve:

$$\max_{w} \quad ll(w) = \max_{w} \quad \sum_{i} \log P(y^{(i)} | x^{(i)}; w)$$

Hill Climbing

A simple, general idea

Start wherever

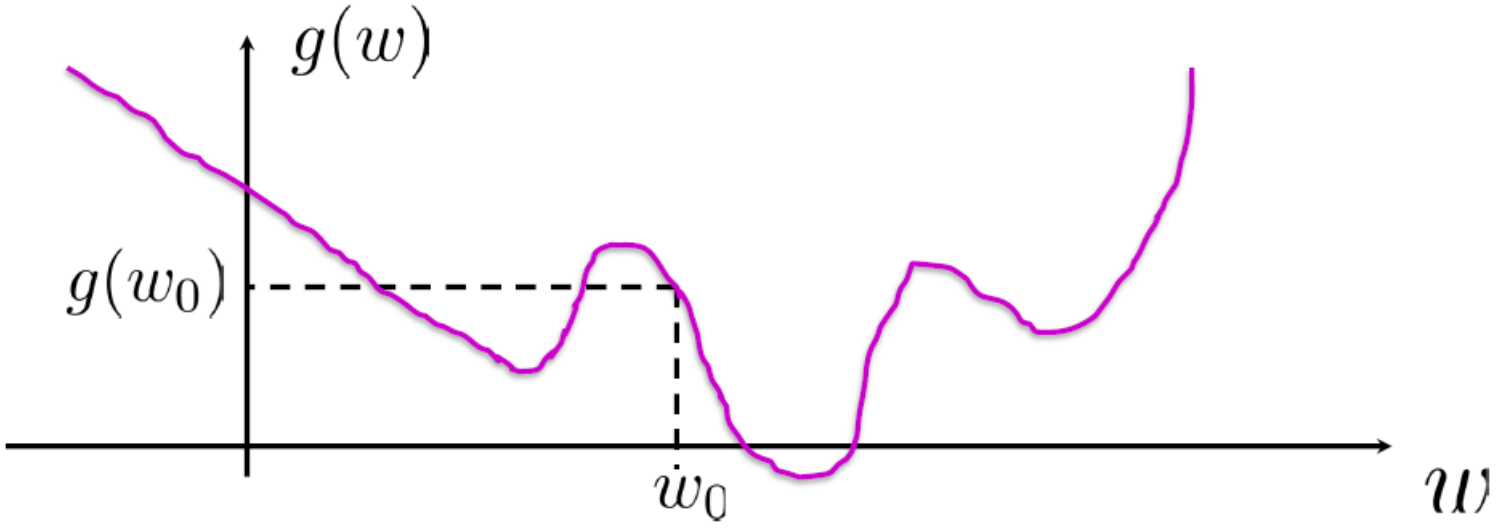
Repeat: move to the best neighboring state

If no neighbors better than current, quit

What's particularly tricky when hill-climbing for multiclass logistic regression?

- Optimization over a continuous space
  - Infinitely many neighbors!
  - How to do this efficiently?

1-D Optimization



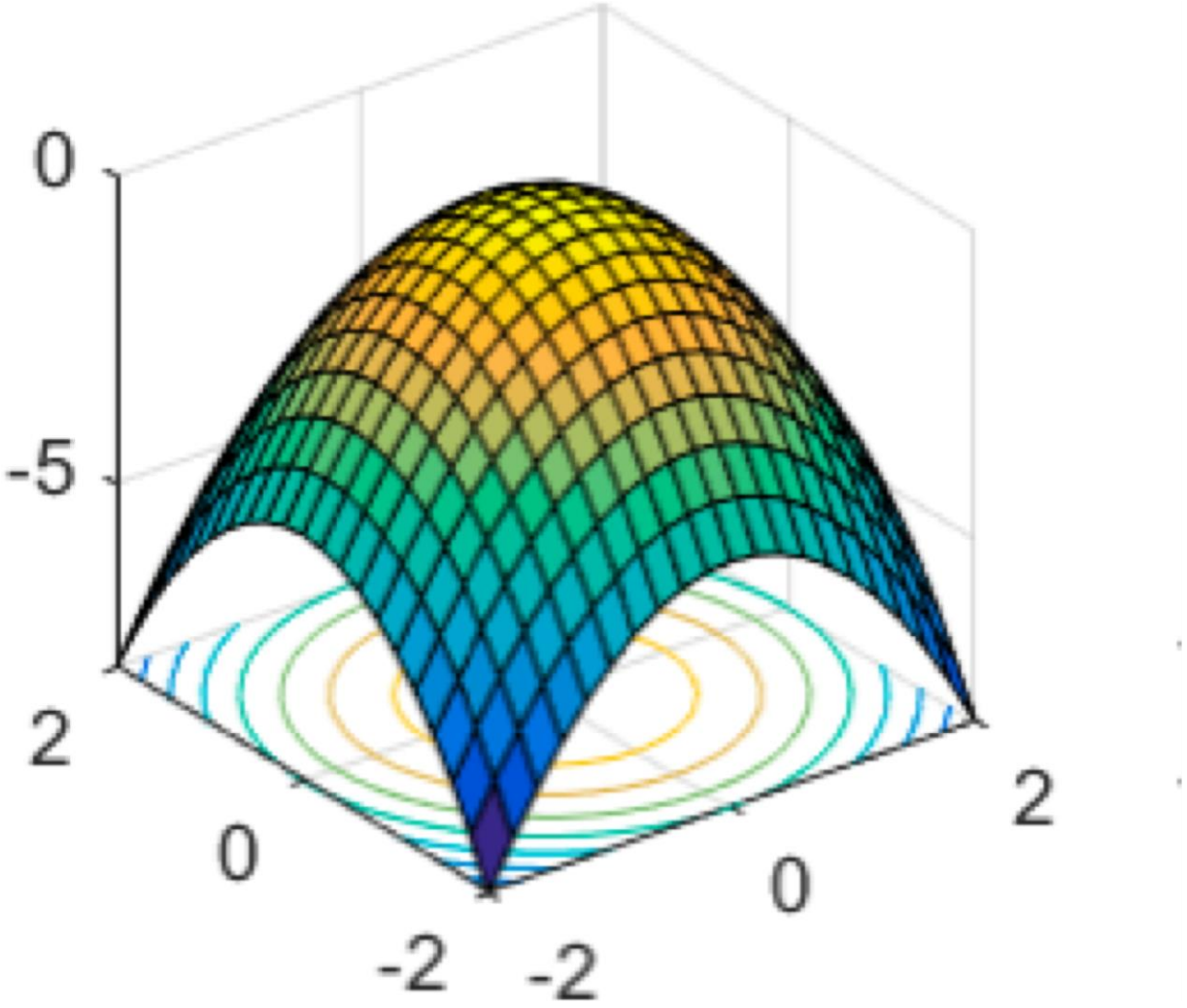
Could evaluate $g(w_0 + h)$ and $g(w_0 - h)$

Then step in best direction

Or, evaluate derivative: $$\frac{\partial g(w_0)}{\partial w} = \lim_{h \to 0} \frac{g(w_0 + h) - g(w_0 - h)}{2h}$$

Tells which direction to step into

2-D Optimization



Source: offconvex.org

Gradient Ascent

Perform update in uphill direction for each coordinate

The steeper the slope (i.e. the higher the derivative) the bigger the step for that coordinate

E.g., consider: $g(w_1, w_2)$

Updates:

$$w_1 \leftarrow w_1 + \alpha * \frac{\partial g}{\partial w_1}(w_1, w_2)$$

$$w_2 \leftarrow w_2 + \alpha * \frac{\partial g}{\partial w_2}(w_1, w_2)$$

- Updates in vector notation:

$$w \leftarrow w + \alpha * \nabla_w g(w)$$

with: $\nabla_w g(w) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(w) \\ \frac{\partial g}{\partial w_2}(w) \end{bmatrix}$ **= gradient**

Steepest Descent

- Idea:
  - Start somewhere
  - Repeat: Take a step in the steepest descent direction

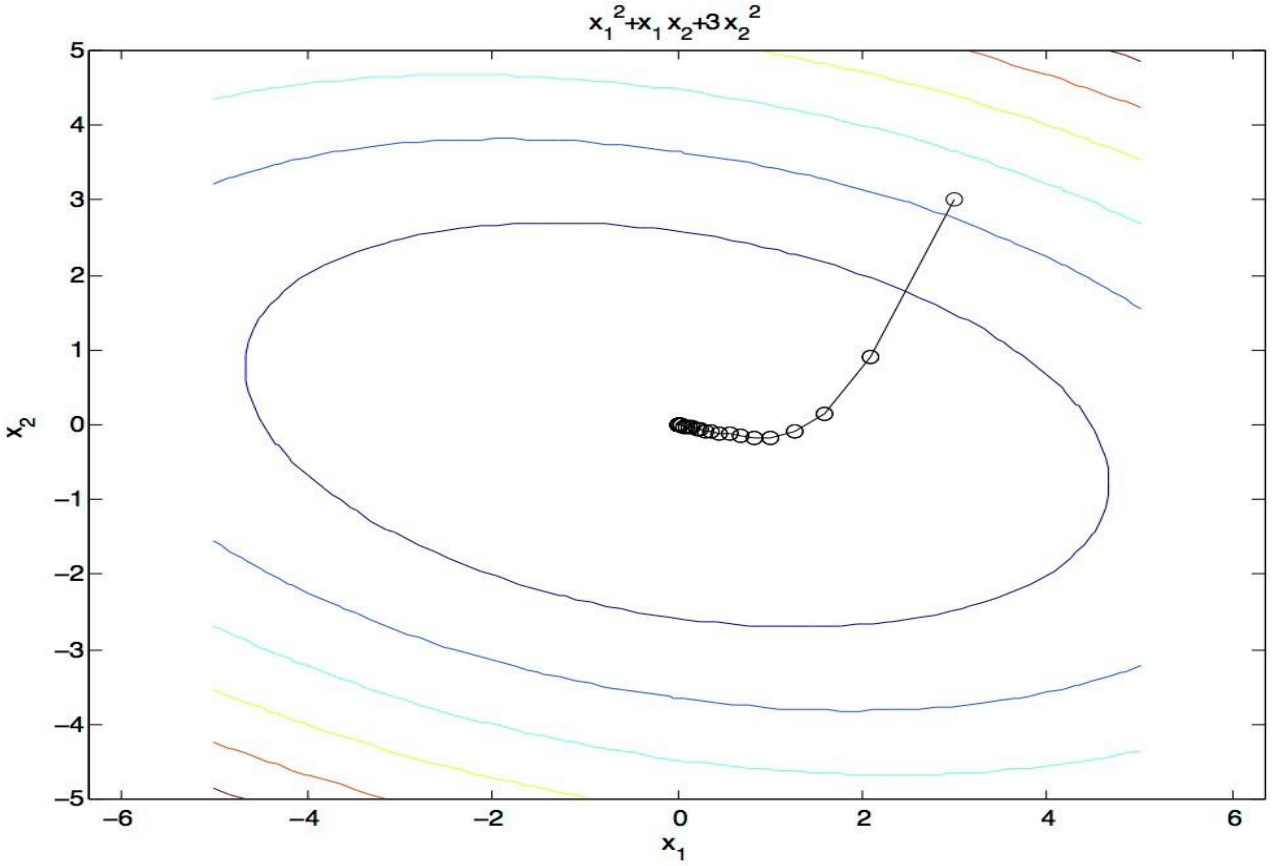


Figure source: Mathworks

Steepest Direction

- Steepest Direction = direction of the gradient

$$\nabla g = \begin{bmatrix} \frac{\partial g}{\partial w_1} \\ \frac{\partial g}{\partial w_2} \\ \cdots \\ \frac{\partial g}{\partial w_n} \end{bmatrix}$$

Optimization Procedure: Gradient Ascent

```
init w
for iter = 1, 2, …
```

$$w \leftarrow w + \alpha * \nabla g(w)$$

- α : learning rate --- hyperparameter that needs to be chosen carefully

Batch Gradient Ascent on the Log Likelihood Objective

$$\max_w \quad ll(w) = \max_w \quad \underbrace{\sum_i \log P(y^{(i)}|x^{(i)}; w)}_{g(w)}$$

```
init w
for iter = 1, 2, …
```

$$w \leftarrow w + \alpha * \sum_i \nabla \log P(y^{(i)}|x^{(i)}; w)$$

Stochastic Gradient Ascent on the Log Likelihood Objective

$$\max_{w} \quad ll(w) = \max_{w} \quad \sum_{i} \log P(y^{(i)}|x^{(i)}; w)$$

Observation: once gradient on one training example has been computed, might as well incorporate before computing next one

```
init w
for iter = 1, 2, …
  pick random j
```

$$w \leftarrow w + \alpha * \nabla \log P(y^{(j)}|x^{(j)}; w)$$

Mini-Batch Gradient Ascent on the Log Likelihood Objective

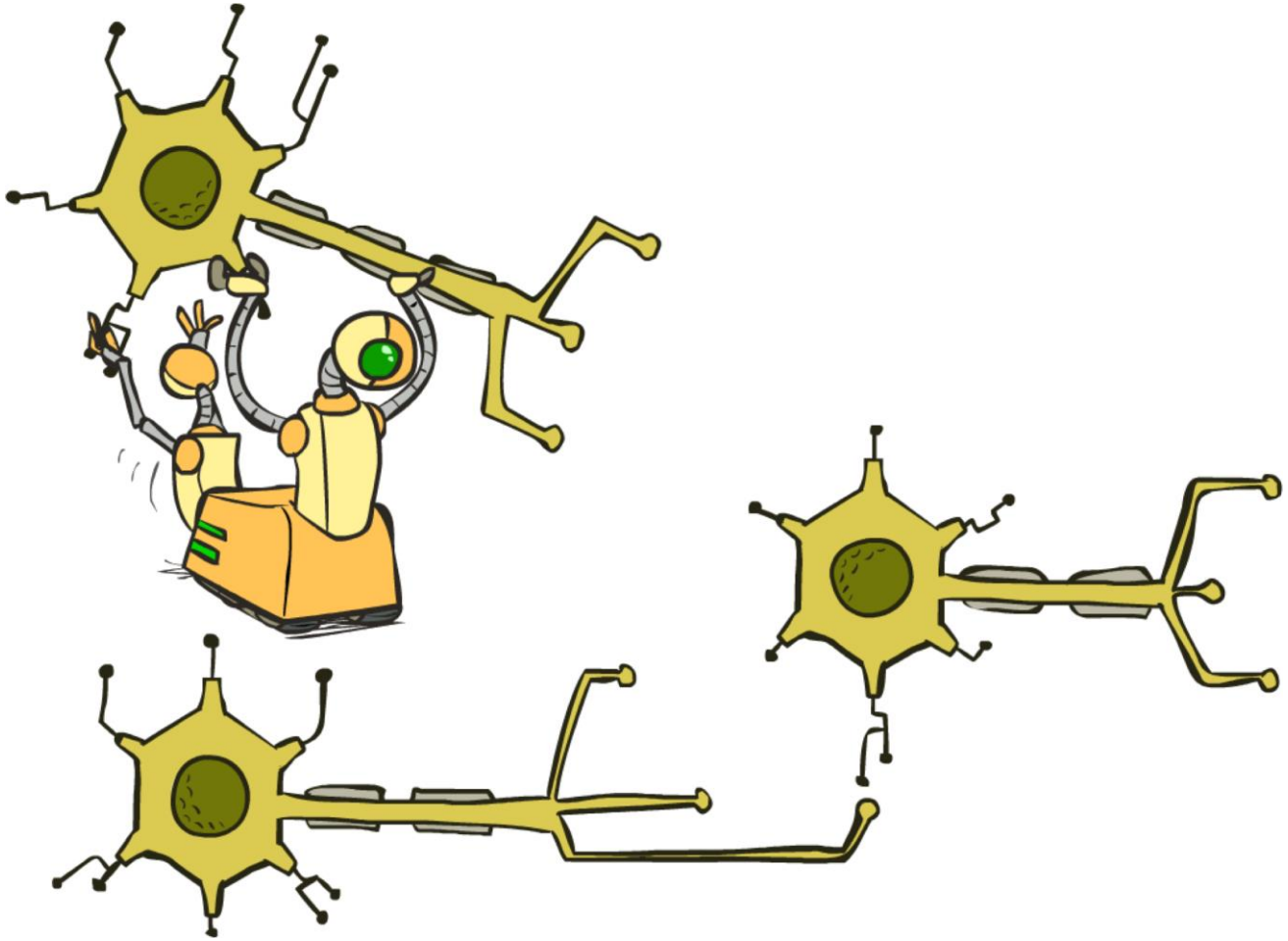
$$\max_w \quad ll(w) = \max_w \quad \sum_i \log P(y^{(i)}|x^{(i)}; w)$$

Observation: gradient over small set of training examples (=mini-batch) can be computed in parallel, might as well do that instead of a single one

```
init w
for iter = 1, 2, …
  pick random subset of training examples J
```

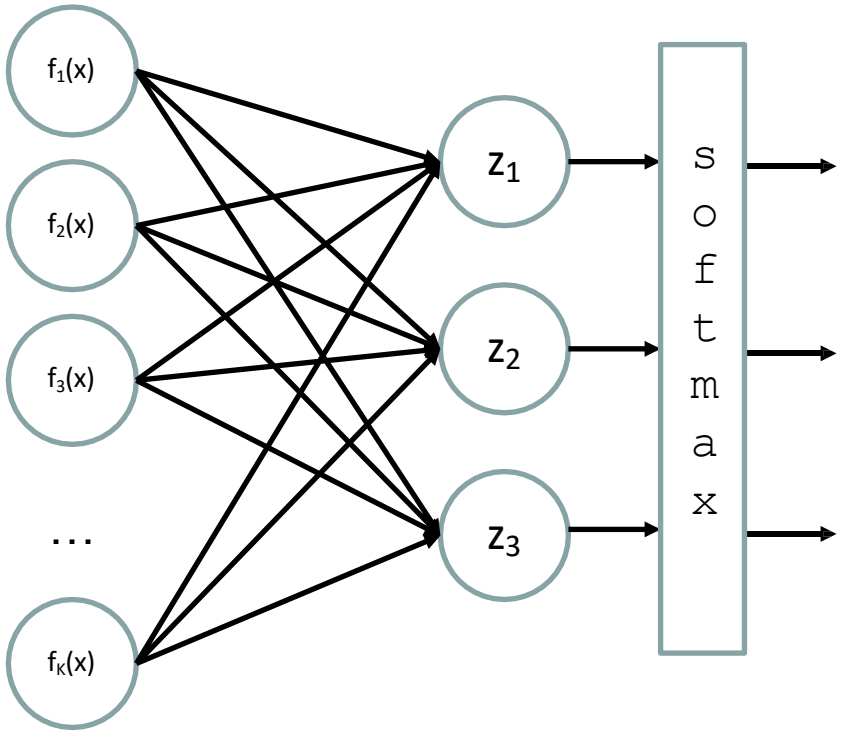
$$w \leftarrow w + \alpha * \sum_{j \in J} \nabla \log P(y^{(j)}|x^{(j)}; w)$$

Neural Networks



Multi-class Logistic Regression

= special case of neural network (single layer, no hidden layer)

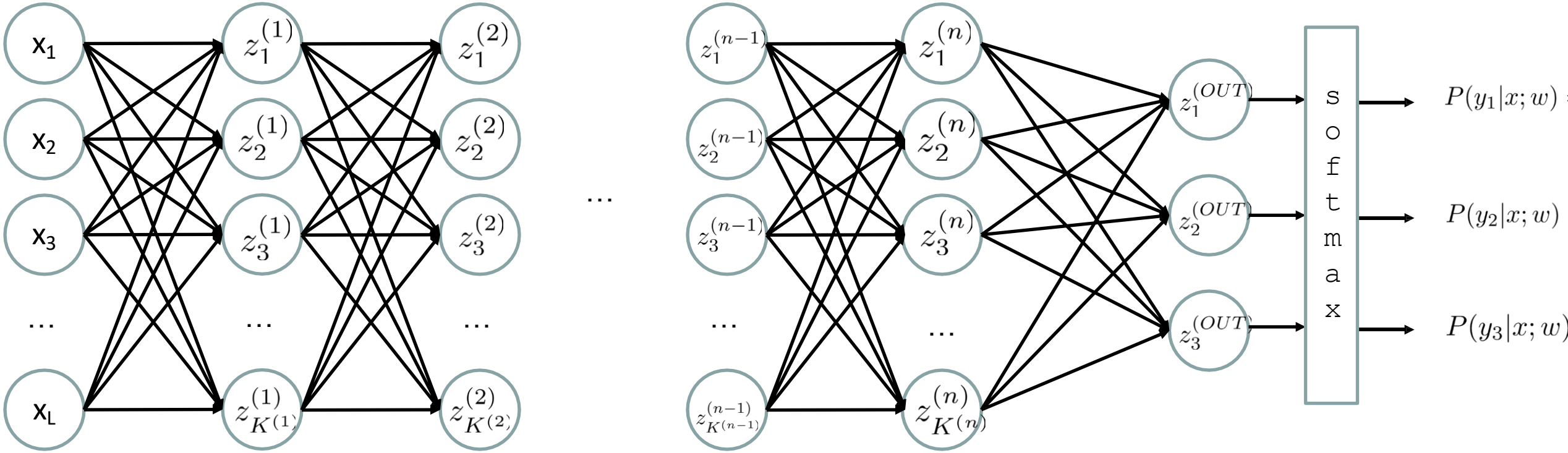


$$P(y_1|x;w) = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$P(y_2|x;w) = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$P(y_3|x;w) = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

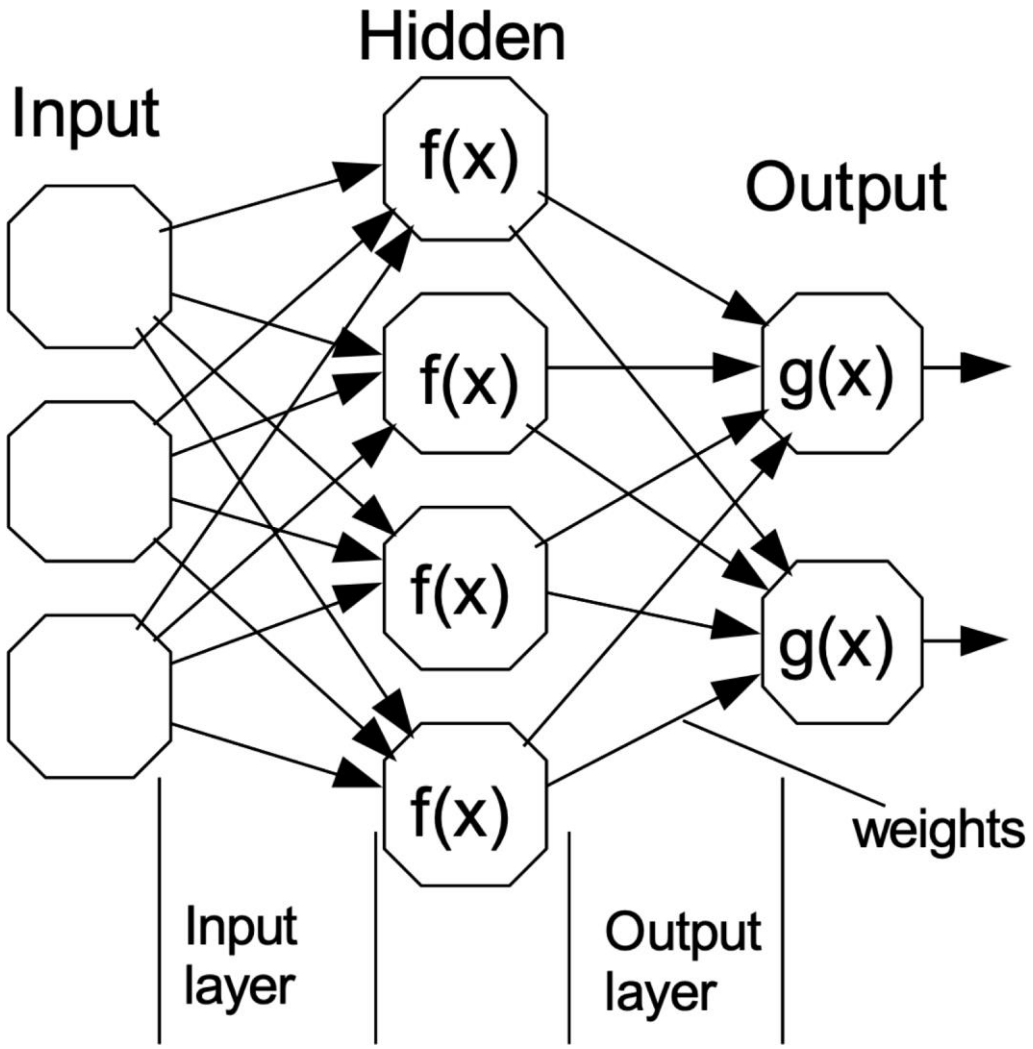
Multi-layer Perceptron



$$z_i^{(k)} = g\Big(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\Big)$$

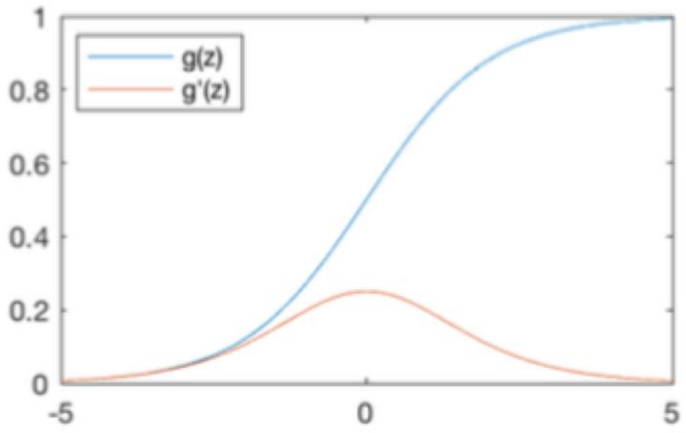
g = nonlinear activation function

Multi-layer Perceptron



Common Activation Functions

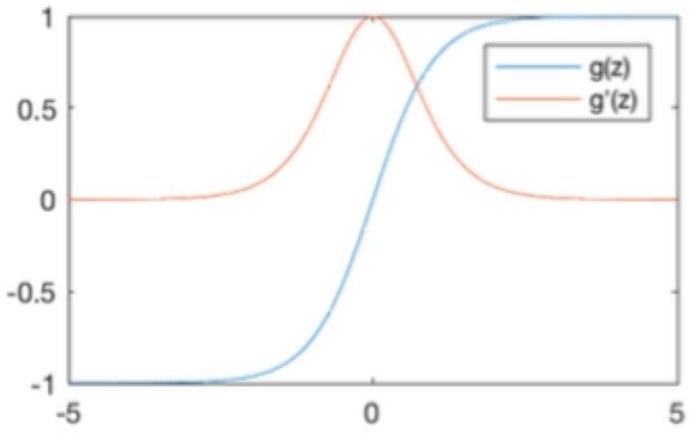
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

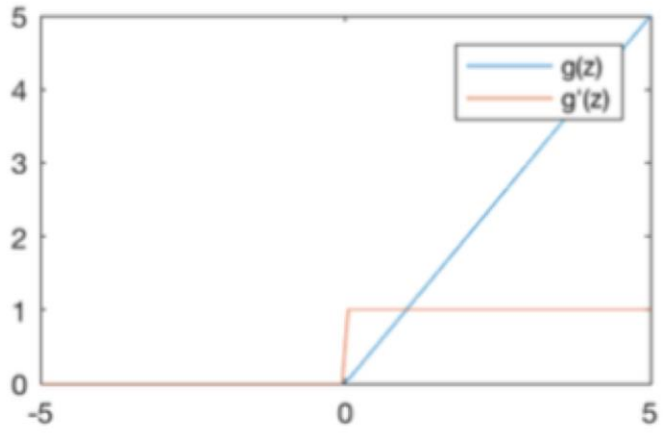
Hyperbolic Tangent



$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

[source: MIT 6.S191 introtodeeplearning.com]

Multi-layer Perceptron

Training the MLP neural network is just like logistic regression:

$$\max_{w} \quad ll(w) = \max_{w} \quad \sum_{i} \log P(y^{(i)}|x^{(i)}; w)$$

just w tends to be a larger vector

just run gradient ascent ➔ **Back-propagation algorithm**

Neural Networks Properties

Theorem (Universal Function Approximators). A two-layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.

Practical considerations

- Can deal with more complex, nonlinear classification & regression
- Large number of neurons and weights
  - Danger for overfitting

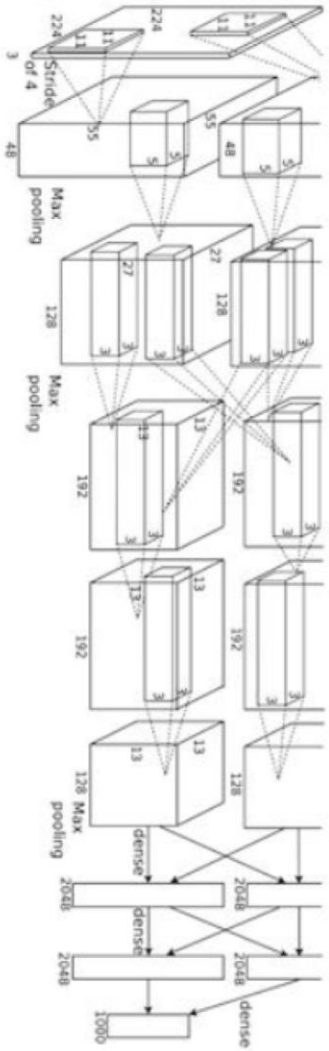
Deep Learning Model

Convolutional network
(AlexNet)

input image

weights

loss



Neural network as
General computation graph

Krizhevsky, Suskever, Hinton, 2012

Deep Learning Model

MIT Technology Review

10 BREAKTHROUGH TECHNOLOGIES 2013

Deep Learning

With massive amounts of computational power, machines can now recognize objects and translate speech in real time. Artificial intelligence is finally getting smart.

Temporary Social Media

Messages that quickly self-destruct could enhance the privacy of online communications and make people freer to be spontaneous.

Prenatal DNA Sequencing

Reading the DNA of fetuses will be the next frontier of the genomic revolution. But do you really want to know about the genetic problems or musical aptitude of your unborn child?

Additive Manufacturing

Skeptical about 3-D printing? GE, the world's largest manufacturer, is on the verge of using the technology to make jet parts.

Baxter: The Blue-Collar Robot

Rodney Brooks's newest creation is easy to interact with, but the complex innovations behind the robot show just how hard it is to get along with people.

Memory Implants

A maverick neuroscientist believes he has deciphered the code by which the brain forms long-term memories. Next: testing a prosthetic implant for people suffering from long-term memory loss.

Smart Watches

The designers of the Pebble watch realized that a mobile phone is more useful if you don't have to take it out of your pocket.

Ultra-Efficient Solar Power

Doubling the efficiency of a solar cell would completely change the economics of renewable energy. Nanotechnology just might make it possible.

Big Data from Cheap Phones

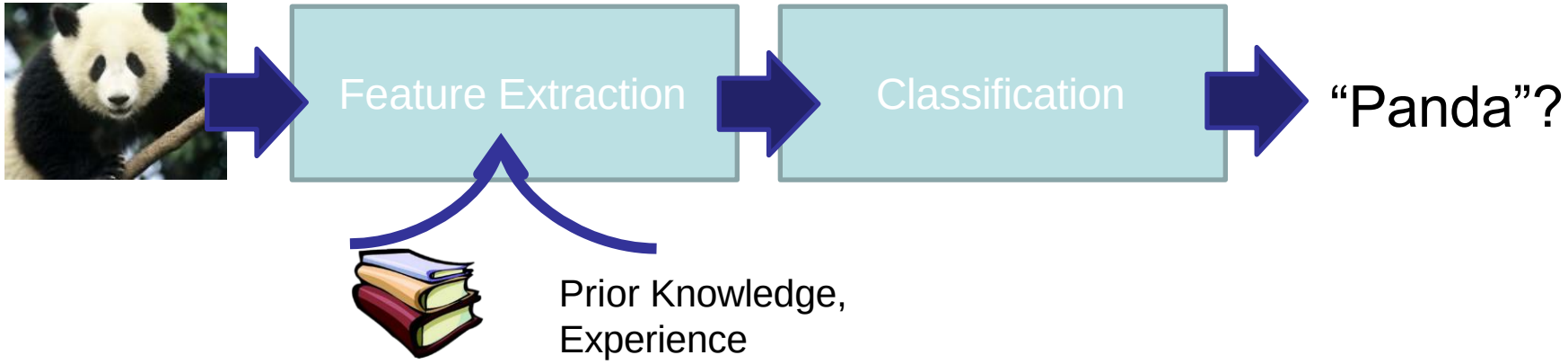
Collecting and analyzing information from simple cell phones can provide surprising insights into how people move about and behave – and even help us understand the spread of diseases.

Supergrids

A new high-power circuit breaker could finally make highly efficient DC power grids practical.

Deep Learning Model

- We need **good** features!



Pose

Occlusion

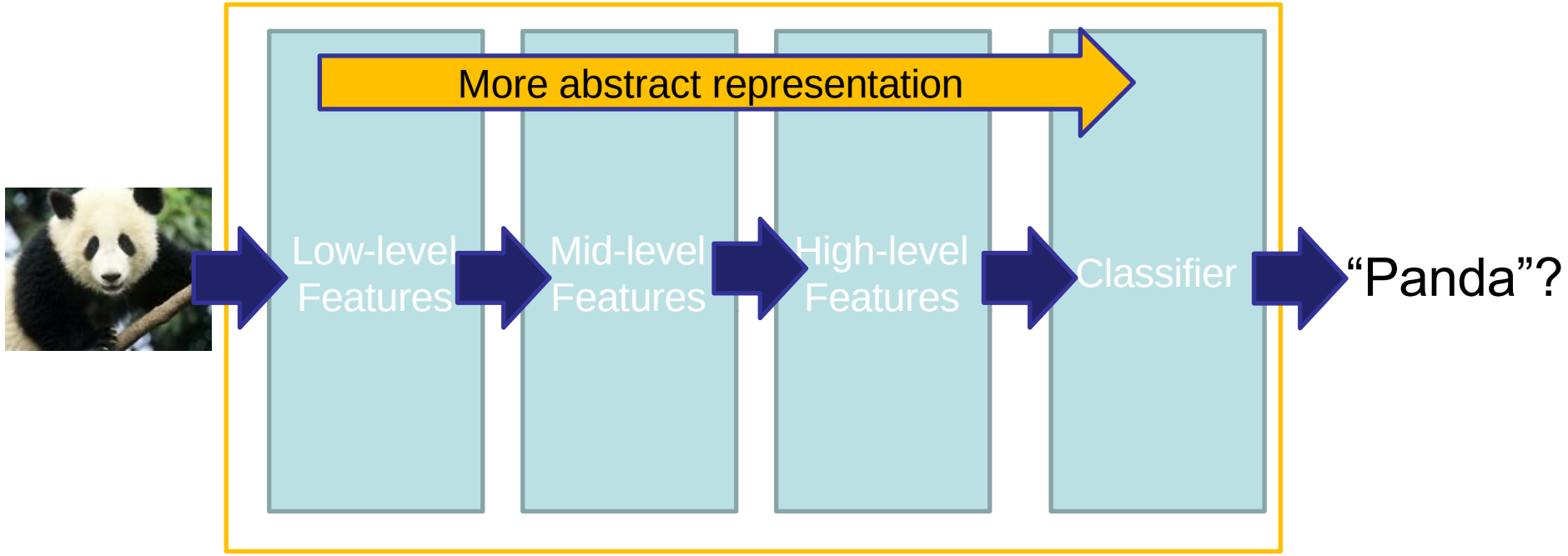
Multiple objects

Inter-class similarity

Image courtesy of M. Ranzato

Deep Learning Model

- **Directly learn** features representations from data.
- **Joint learn** feature representation and classifier.



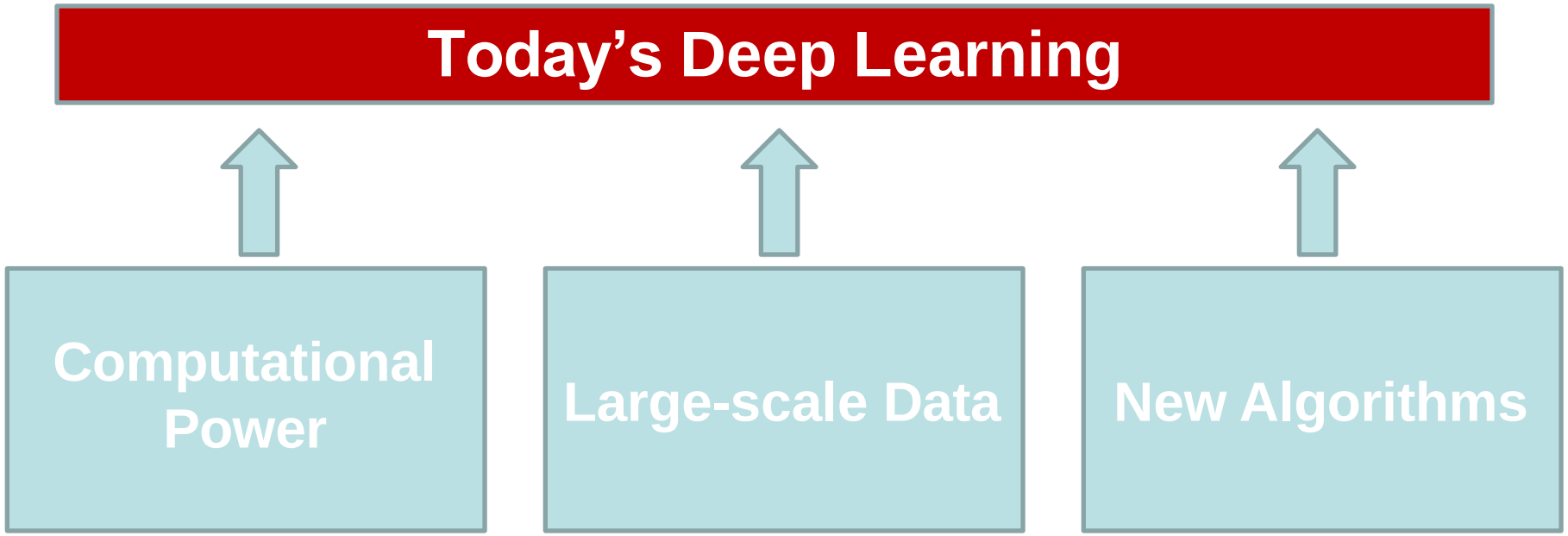
Deep Learning: train layers of features so that classifier works well.

Image courtesy of M. Ranzato

Deep Learning Model

Have we been here before?

- Yes.
  - Basic ideas common to past **neural networks** research
  - Standard machine learning strategies still relevant.
- No.



Deep Learning Model

Convolutional Neural Networks (CNNs)

- A special **multi-stage architecture** inspired by visual system
  - Higher stages compute more global, more invariant features

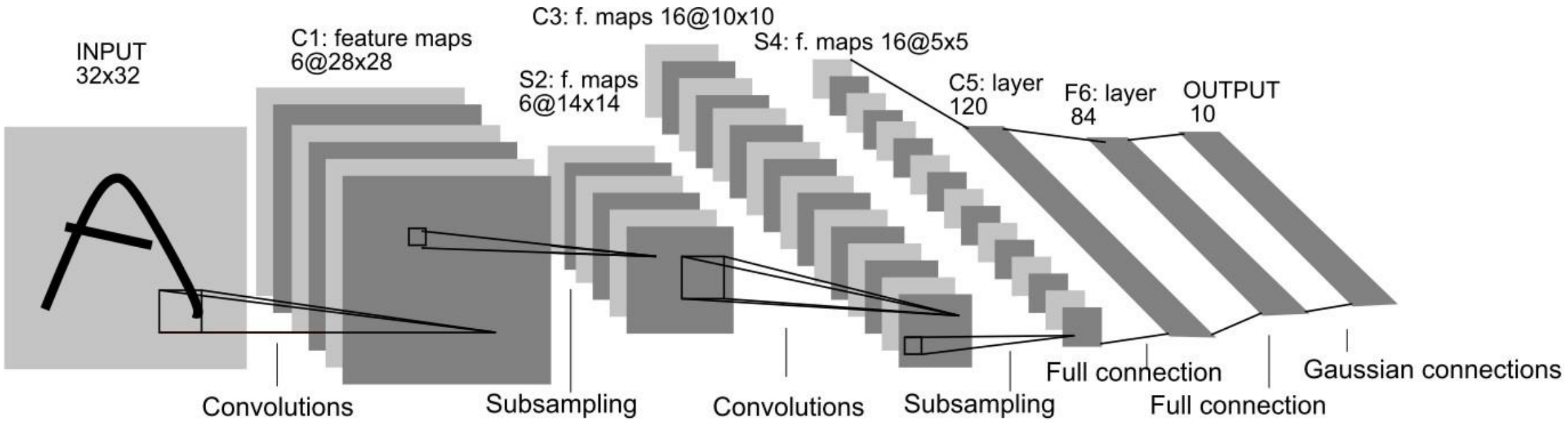


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Deep Learning Model

Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	32x32	-	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

https://www.datasciencecentral.com/lenet-5-a-classic-cnn-architecture/

Different Neural Network Architectures

- Exploration of different neural network architectures
  - ResNet: residual networks
  - Networks with attention
  - Transformer networks
- Neural network architecture search
- Really large models
  - GPT2, GPT3
  - CLIP

Acknowledgement

The lecture slides are based on the materials from ai.Berkey.edu

Questions?

UNIVERSITY
OF WOLLONGONG
AUSTRALIA